CSC_51053_EP, Lab Session: Postgres

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

10 October 2025

Outline

In this lab you will install and start using PostgreSQL (or Postgres, in short), a widely-used, robust and open source relational database management system.

`psql` : one of the postgres executables

`mkdir` : UNIX terminal instruction

`COPY` : SQL/psql instruction

Instruction: what you must do

Explanation or troubleshooting.

Figure 1: Color codes used in this document.

1 Installation (of the Postgres software or app)

- **Postgres may already be installed on your machine**; start by searching for it. Other versions can also be used.
- **Postgres will not run as a process of the root user** (it's considered a security risk). Thus:

  Identify an operating system user on your machine that is not root. That user should install Postgres.

- Download Postgres **v14** or a more recent one from `https://www.postgresql.org/download/`. Any installation creates a **postgresql** directory.

  While doing the installation: (1) keep track of (write somewhere) the full path to this directory; (2) if offered the opportunity, install also pgAdmin, a Graphical User Interface (GUI) for a Postgres server.

- Under **postgresql/bin** there are several executable programs; we will refer to each such executable by writing these instructions `this way`.

Below, we provide commands that should work on a Linux (or MacOS) console. They can be adapted/customized slightly from one machine to another.

Online documentation can be found at:
`https://www.postgresql.org/docs/14/static/index.html`;
`https://docs.postgresql.fr/` is also a useful (French-only) portal.
`http://www.stackoverflow.com` documents *many* common incidents and their solutions.

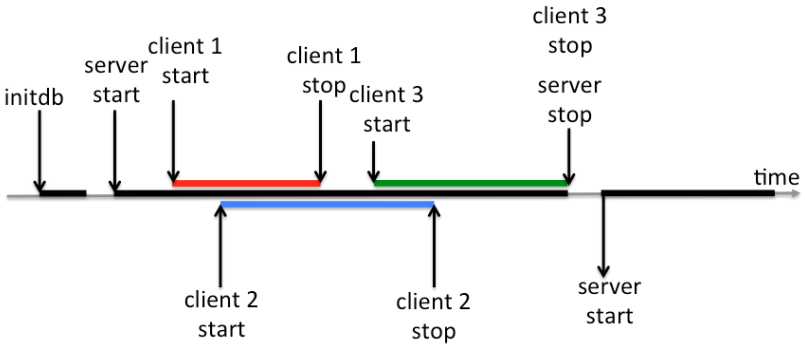
In this lab description, we will use **PSQLPath** as an abbreviation complete path to the directory holding the **executable** files that are part of Postgres, e.g., `/usr/lib/postresql/14/bin/`.

You may call these executables by typing the complete path, then adding the executable name. Alternatively (preferred), you can use only the name of the executable; to do so, you need to modify the environment variable `$PATH` by appending to the `.bashrc` file in your home directory the following instruction:

```
export PATH=/usr/lib/postgresql/bin:$PATH
```

We recommend to create a directory for storing Postgres' data; the full path from the root of the file system to this directory is denoted below **DATAPath**. A second (or the same) directory may be used to store Postgres' execution log; we will denote the corresponding full directory by **LOGPath**.

2 Lauching the Postgres server and client



Postgres (like the other major relational database engines) follows a **client-server** model.

2.1 The server

To work on a database, a database **server** needs to be running. A server manages a **disk area (also called cluster)** which has been previously initialized using `initdb`.

Initialize a Postgres disk area there by running:
`PSQLPath/initdb -D DATAPath -E UTF8`

`initdb` allocates the storage that the Postgres server will use to store one or many databases, including the default database which is called `postgres`.
The `-D` parameter is the directory where the disk area (cluster) is. There may be several such disk areas on a computer; each needs to be created with `initdb`.
The `-E` parameter ensures that within the database, characters are encoded in bytes in the UTF8 model.

Launch the server by running:
`PSQLPath/pg_ctl -D DATAPath -l LOGPath/logfile start`

This instruction starts the server proces, which will run in the background; you now have a server running.
The `-D` parameter is the disk area to be used by the server.
The `-l` parameter indicates the file where the server will log its activity.

We keep the server running during the lab. When you shut it down, data remains on disk and can be used by a server in the future.
If you try to start a server while one is already running, the second attempt will fail, because the first server locks a resource (network port 5432 by default) that the second server cannot get. If you see such an error message, it usually means that one server is already running.

Once it is running, give a look to the log file `LOGPath/logfile` to get familiar with the format.

Within a disk area, different databases are separated from each other. Each application usually works on one database.

2.2 The clients

Database operations are performed through a **client** which connects to a server. The **the interactive client** (`psql`) allows to open a session and execute a sequence of commands, before closing a session. **Graphical interactive clients** also exist.

Postgres maintains its own set of access rights to database objects through **roles**; a role can be seen as Postgres' equivalent for a user or group. Usually, a **Postgres role called postgres** (password: postgres, or empty password) is created automatically for each server, having all the rights on this cluster. Some installation procedures also allow you to **chose** the name and password for this first role. If so, **write them down**!

Connect to the server at database postgres using role $ROLE through an interactive client:
`psql -U $ROLE -d postgres`

Here, `$ROLE` is the Postgres role mentioned above.

Useful commands for the interactive client psql:

- **\conninfo** or **\c** shows information about the current connection
- **\list** or **\l** lists the available databases
- **\d** lists the tables and views in the current database
- **\d table** lists all details related to the relation table (integrity constraints etc.)
- **\dt table** lists the signature of the relation **table** (its attributes and their types)
- **\h command** gives help (usage) information about **command**
- **\i myfile.sql** executes a given script
- **\?** shows all the available shorthand commands
- **\q** or **exit** ends the connection (exits the client).

Understanding the psql prompt:

```
nombase->
nombase=#
```

the active database is *nombase*

= : DBMS is waiting for new instruction

- : DBMS is waiting for the end of SQL instruction (ends with ;)

(or ' or * : DBMS expects closing parenthesis or others...

/ > : you are/are not the DB Administrator

3 Creating the database schema, loading the data

In this section, you will create and manipulate a database from scratch.

3.1 First, tiny database: government positions

Create a database called govt. This can be done using the `createdb` executable, or (in the interactive client) with CREATE DATABASE govt. Then you may connect to the database from the psql client using \connect govt

1. Create and connect to the database called govt.
2. Using the instructions in the `govt-database.txt` file in Moodle, create the schema and load the data in your database.

   On this database, write in SQL the queries given there in natural language (1. to 6.)
3. You may then drop the database with DROP DATABASE govt.

   If you tried this instruction while connected to the govt database, Postgres probably returned an error message indicating that it is forbidden to do so. One solution: connect to the default postgres database and drop govt from there.

3.2 Second, small database: users and gift preferences

Create a database called smallgifts, then connect to it.

Download the file `small-gifts.zip` provided in Moodle, and unzip the archive.

```
unzip small-gifts.zip
```

You will get a directory containing a file `gift.csv` with the data for a relation gift, together with a `readme.txt` file that describes the fields. As you can see in Figure 2, `gift.csv` is in csv (comma-separated values) format, and has no header.

```
38717,samsung rs68n8240b1,6876,5
42575,smeg fab30rp1 fridge freezer,7620,5
42576,liebherr icus3214 comfort,6824,5
42579,liebherr icuns3314 comfort,5604,5
42580,smeg fab32rnen1 fridge freezer,7656,5
```

Figure 2: The first 5 lines of the file `gift.csv`.

1. Create a table called gift, with attributes that should allow it to store the data from the file `gift.csv`.
2. Then load the data using the `psql` instruction:

   COPY gift FROM '/my/path/to/.../gift.csv' CSV DELIMITER ',';

COPY requires an *absolute path to the file* (replace it as appropriate on your computer). Postgres also provides a `copy` executable, with a similar role as the COPY instruction understood by `psql`.
Windows users If COPY returns an access right error but the read access to the data file is sufficiently broad, try \COPY instead.
Sometimes the CSV has a headers, in which case the HEADER option informs Postgres that the first line of the file is header (as opposed to data):
COPY gift FROM 'gift.csv' CSV HEADER.

If, in the CSV file, the delimiter is ';' the loading instruction should read:
COPY gift FROM 'gift.csv' CSV DELIMITER ';' HEADER.
For more options, type \h copy in the interactive client.

1. What is the result obtained with the instructions below?

   ```
   ALTER TABLE gift ADD CONSTRAINT pk_id PRIMARY KEY(id);
   ALTER TABLE gift ADD CONSTRAINT uc_name UNIQUE(descr);
   ```

2. Try to insert a tuple in the database with giftid 42575, for instance:

   ```
   INSERT INTO gift VALUES (42575, 'Samsung TV', 6788, 5);
   ```

3. Write in SQL the following simple queries on gift, and get them evaluated by the server:
   - Get the item ids and their description whose price is greater than 7000.
   - Display the number of gift items in the gifts table.

3.3 Access control

Access rights are handled by Postgres as follows.

Several Postgres users (roles) could use a given database. Postgres calls **privilege** the right of a role to perform a given action on a database, or a specific object (table, view etc.) in a database. Postgres privileges include:

- SELECT: the right to ask a query over a table or view
- INSERT: the right to insert data in a table
- UPDATE: the right to modify the data in the table
- CREATE: the right to create tables in a database
- GRANT: the right to give access rights (to the respective database, table, view etc.) to other users
- DELETE: the right to delete a tuple from a table
- ALL is used a shorthand for all possible privileges.

Create another Postgres user (role):
`CREATE USER myrole;`
Create a view defined by an SQL query, e.g., one of the queries you wrote above.
Grant the user the right to query that view:
`GRANT SELECT ON viewname TO myrole;`
Check the result by connecting to the database as the second user (for instance, in a second terminal, in parallel to the first client session).

4 Complex Queries

The goal of this section is to write more complex queries. For this, you will use a database similar to the smallgifts database above, but with more tables and more data. The database is extracted from a real-world gifts dataset. Figure 3 shows the schema of the database[1].

[1] This graph was generated by pgAdmin4. Using pgAdmin: connect to a server, then select the database and choose 'ERD for Database' to obtain a diagram of the database schema. Note the symbols used for primary and foreign keys.

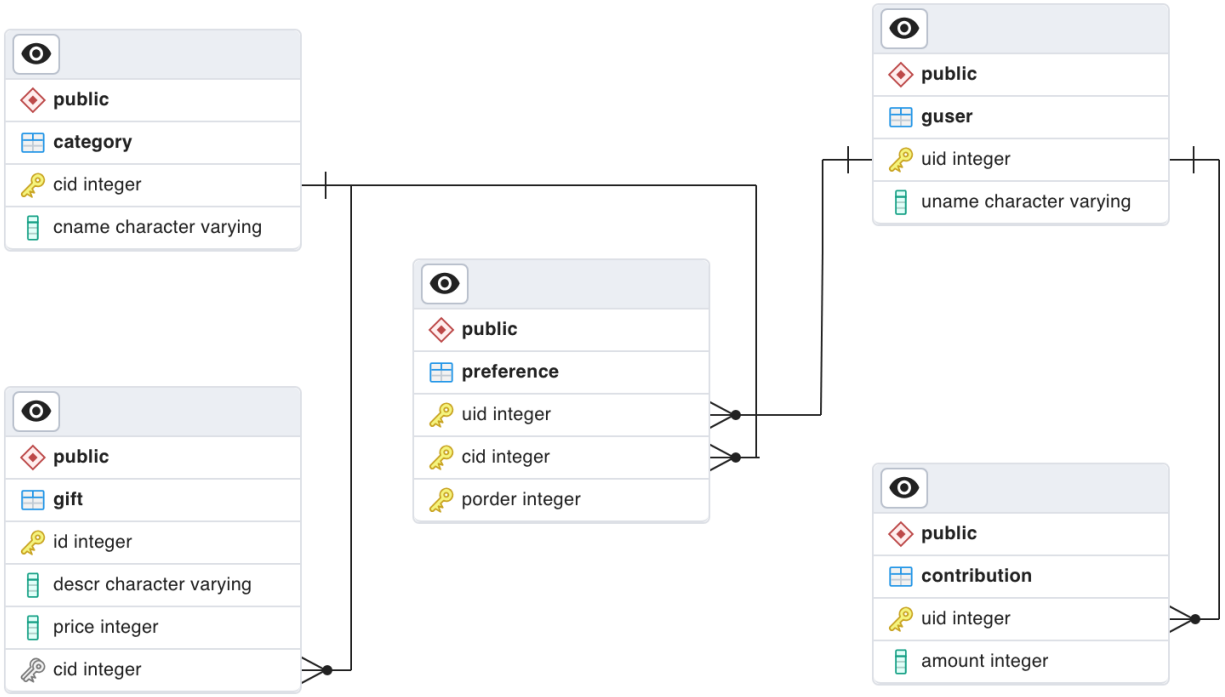


Figure 3: Gifts Schema

Description of the database

The tables have the following schema:

create table **guser**(uid int primary key, uname varchar);
– *people who pool money to buy a gift*
create table **category**(cid int primary key, cname varchar);
– *gift categories, e.g., 'sport', 'art', 'cooking' etc.*
create table **gift**(id int primary key, descr varchar, price int, cID int references category(cid));
– *possible gifts that could be bought*
create table **contribution**(uid int primary key references guser(uid), amount int not null);
– *each user may contribute (or not) an amount of money to the gift*
create table **preference**(uid int references guser(uid), cid int references category(cid), porder int not null check (1<=porder and porder<=3), primary key (uid, cid, porder));
– *each user may specify at most 3 preferences for the category of the gift to be bought. A user preference order (porder) of 1 is the option the user favors most; 2 is the "next" preference order, and 3 is the least favored preference order*

If a user u put a category c first in her preference order (porder=1), we view this as a *support score* of 3; similarly, porder=2 means a support score of 2, while porder=3 means a support of 1.

Create a new database called giftsdb.
Download from Moodle the dump of the database (dump_gifts.sql); the command

```
psql -d giftsdb -f dump_gifts.sql
```

will populate it with data. On this new database, write and evaluate the following SQL queries.

1. Write the SQL query Q1: for each category whose name starts with 'M', with non-zero total support across all users, find the category name and the total category support. Order the categories from the highest to the lowest total support.
2. Write the variant query Q2: for each category (including those with no support), find the category name and the total support. Provide one variant using an outerjoin, and another without.
3. A gift is *affordable* if its price is less than the sum of all contributions. Write the query Q3: For each affordable gift, return the gift id, description and price.

5 To shut down your server!

Shutting the server down

```
PSQLPath/pg_ctl -D DATAPath -l LOGPath/logfile stop
```

When this is finished, look in the server's log file.

Server survival

Putting a laptop to sleep **does not** shut down the server.
Rebooting the laptop **does** shut it down. (You will need to start it again in order to use Postgres.)